

Computer Organization
Reference Guide

Study workflow: identify the relevant topic, apply the matching pattern, then substitute values. Show method: formula -> substitution -> result. State assumptions for invented ISA questions. For CPU speed, never conclude from GHz alone: $T = I_c \cdot \text{CPI} / f = I_c \cdot \text{CPI} \cdot \text{cycle_time}$, $\text{CPI} = \text{sum}(\text{CPI_i} \cdot I_i) / I_c$, $\text{MIPS} = f / (\text{CPI} \cdot 10^6)$. MIPS/MFLOPS not reliable across ISAs; real benchmark time is better.

Digital Logic

Use this when asked: truth table to expression, Boolean laws, NAND/NOR/XOR, or "show gates are functionally complete."

Truth table -> expression: for every row with output 1, write one minterm. Use variable if bit=1, complement if bit=0. OR all minterms. Simplification not required unless asked.

Laws: De Morgan: $\text{not}(A \cdot B) = \text{not } A + \text{not } B$, $\text{not}(A + B) = \text{not } A \cdot \text{not } B$. XOR=exactly one 1.

Functional complete: AND+NOT can build OR: $A+B = \text{not}((\text{not } A) \cdot (\text{not } B))$. Thus AND, OR, NOT are possible.

Radix + Signed

Use this when asked: convert bases, write bit patterns for positive/negative numbers, find range, or interpret a bit pattern.

Value: $(\dots a_2 a_1 a_0 . a_{-1} \dots)_r = \text{sum } a_i \cdot r^i$.

Decimal -> base r int: repeatedly divide by r; remainders read upward. Fraction: repeatedly multiply by r; integer parts read downward.

Binary/hex: group 4 bits from radix point. Prefer hex for long binary answers.

Scheme, w bits	Encode / decode	Range
Unsigned	V=P	$0 \dots 2^w - 1$
Sign-mag	MSB sign, rest magnitude	$-(2^{w-1} - 1) \dots + (2^{w-1} - 1)$, two 0s
1's comp	$\text{¬}N = \text{invert}(+N)$; if MSB=1, $V=P - (2^{w-1})$	same as sign-mag, two 0s
2's comp	$\text{¬}N = \text{invert}(+N) + 1 = 2^w - N$; if $P \geq 2^{w-1}$, $V=P - 2^w$	$-2^{w-1} \dots + (2^{w-1} - 1)$
Excess-k	stored $P = V+k$; $V=P-k$	$-k \dots 2^w - 1 - k$

2's addition: add bit patterns; discard carry out of MSB. Overflow only when same-sign inputs give opposite-sign result.

1's addition: add; if carry out, add 1 back (end-around carry).

Sign extension 2's: copy old sign bit into new upper bits.

Floating Point

Use this when asked: encode/decode S/E if formats, find bias, largest/smallest value, or explain FP add/mul/div. First write the format and bias.

Generic: $V = (-1)^S \cdot 1.f \cdot 2^{(E-b)}$. Bias for m exponent bits: $b = 2^{m-1} - 1$.

Encode decimal x: sign -> convert |x| to binary -> normalize as $1.f \cdot 2^e$ -> stored exponent $E = e + b$ -> store fraction bits after hidden 1; pad/truncate.

Decode hex: hex -> binary -> split S/E/f -> compute $E - b$ -> evaluate $1.f \cdot 2^{(E-b)}$.

No special exponent patterns	Answer
Smallest positive	$1.0 \cdot 2^{(0-b)}$
Largest positive	$(2 - 2^{-(E-bits)}) \cdot 2^{(2^{(E-bits)} - 1 - b)}$
Increase range	use more exponent bits, fewer fraction bits -> less precision

If IEEE special patterns are considered, exponent all-0/all-1 are special; valid normalized exponents are $1 \dots 2^{m-2}$. Include these details only when explicitly required.

FP add/sub: check zero; align smaller exponent by shifting significand right; add/sub significands; normalize/round. If significand becomes $10.x$, shift right 1 and increment exponent.

FP mul: $\text{sign} = S_1 \text{ xor } S_2$; $E = E_1 + E_2 - b$; multiply significands; normalize/round; check overflow/underflow.

FP div: $\text{sign} = S_1 \text{ xor } S_2$; $E = E_1 - E_2 + b$; divide significands; normalize/round.

Cache Mapping

Use this when asked: set/tag/offset, same-block addresses, direct vs set-associative vs fully associative, or cache final contents.

Always compute in bytes if memory is byte-addressable.

$\text{block_bytes} = \text{block_words} \cdot \text{bytes_per_word}$

$\text{cache_lines} = \text{cache_size} / \text{block_size}$

$\text{sets} = \text{cache_lines} / \text{ways}$

$\text{block\#} = \text{floor}(\text{addr} / \text{block_bytes})$, $\text{offset} = \text{addr} \bmod \text{block_bytes}$

Org	Index	Tag
Direct	$\text{line} = \text{block\#}$ mod lines	$\text{floor}(\text{block\#} / \text{lines})$
k-way	$\text{set} = \text{block\#} \bmod \text{sets}$	$\text{floor}(\text{block\#} / \text{sets})$
Fully assoc	none	block#

Bits: offset $\text{bits} = \log_2(\text{block_bytes})$. Direct: line $\text{bits} = \log_2(\text{lines})$. k-way: set $\text{bits} = \log_2(\text{sets})$. Tag=remaining upper bits.

Same block addresses: $\text{start} = \text{addr} - \text{offset}$, $\text{end} = \text{start} + \text{block_bytes} - 1$.

Tracing: for each block: map to set/line; hit if present. Miss if absent. If full, evict FIFO oldest or LRU least recently used. Direct map has no choice.

Cache Timing

Use this when asked: hit rate, miss rate, miss penalty, AMAT, or multi-level cache timing. Keep units as ns/ms.

$\text{AMAT} = \text{hit_time} + \text{miss_rate} \cdot \text{miss_penalty}$

$\text{hit_rate} = 1 - \text{miss_rate}$

Course trace style: if each listed block has avg R references, $\text{total_refs} = \text{listed_blocks} \cdot R$, $\text{hit_rate} = 1 - \text{misses} / \text{total_refs}$.

Penalty if one word first then later words: $\text{first} + (\text{block_words} - 1) \cdot \text{later}$. If memory transfers g words at a time: $\text{groups} = \text{ceil}(\text{block_words} / g)$, $\text{penalty} = \text{first_group} + (\text{groups} - 1) \cdot \text{later_group}$.

Multi-level: $L_1 + MR_1 \cdot (L_2 + MR_2 \cdot (L_3 + MR_3 \cdot \text{Mem}))$.

Write-through: write cache and memory; consistent but slower/needs buffer.

Write-back: write memory only on replacement if dirty; faster but memory inconsistent.

Locality: temporal = recently used likely reused. Spatial = nearby addresses likely used. Row-major C arrays: make last subscript the inner loop.

Split cache: instruction cache + data cache. Reduces structural/resource hazard between instruction fetch and data access; does not solve data/control hazards.

Disk, RAID, SSD

Use this when asked: rpm, average rotational latency, sector timing, RAID reliability/capacity, or whether SSD is suitable for page file.

Disk access = seek + rotational delay + transfer.

$\text{rev_time_ms} = 60000 / \text{rpm}$; average rotational latency = $\text{rev_time} / 2 = 30000 / \text{rpm}$.

$\text{sector_time} = \text{rev_time} / \text{sectors_per_track}$.

Consecutive n sectors same track: seek + avg_latency + n * sector_time. Scattered n sectors: $n \cdot (\text{seek} + \text{avg_latency} + \text{sector_time})$.

RAID	Key point
0	striping, no redundancy, fastest, one disk fail loses data
1	mirroring, high reliability, reads can use either copy, writes duplicated
3/4	single dedicated parity disk; recover 1 failure; parity bottleneck
5	distributed parity; survives 1 disk failure; capacity = $(N-1) \cdot \text{disk}$
6	two distributed parity values; survives 2 disk failures; capacity = $(N-2) \cdot \text{disk}$

SSD for page file: suitable vs HDD due low access time and random I/O, but still much slower than RAM; heavy writes cause wear; more RAM is better.

Virtual Memory

Use this when asked: page fault, TLB, logical vs physical address, demand paging, or page file.

Logical/virtual address is program-visible; physical address accesses RAM. MMU maps virtual page number -> physical frame number; offset copied unchanged. PTE contains valid bit V, protection P, dirty D, physical frame number. Page fault: locate page on disk -> find/free/evict frame -> if victim dirty write back -> invalidate old PTE -> load page -> update PTE -> restart process. TLB caches page-table entries.

Addressing + ISA

Use this when asked: define addressing modes, translate $10(R_1)$, explain instruction format, or hand-assemble simple instructions.

Mode	Meaning	Example
Immediate	operand=A	MOV #200, R6
Direct	EA=A, operand=mem[EA]	LD A, R1
Indirect	EA=mem[A], operand=mem[EA]	LD (A), R1
Register	operand=Rval	MOV R1, R2
Reg indirect	EA=Rval, operand=mem[EA]	LD (R1), R2
Displacement	EA=A+Rval, operand=mem[EA]	LD 10(R1), R2
Stack	EA=top of stack	PUSH/POP

Rval means the contents of register R. Destination is last in course assembly: ADD R1, R2, R3 means $R3 = R1 + R2$. Branch uses condition flags from previous ALU op.

Opcode	Instruction	Opcode	Instruction
00	ADD	07	ST
01	SUB	08	Bcc
02	NOT	09	HLT
03	AND	0A	PUSH
04	OR	0B	POP
05	MOV	0C	CALL
06	LD	0D	RET

32-bit instruction fields: op s1 s2 d. LD/ST/Bcc use 2-word format; absolute address is next word; addressing-mode byte=FF. Bcc cc: 00 BR, 01 BZ/BEQ, 02 BNZ/BNE.

Micro-ops

Use this when asked: "data movement/transformation sequence" or datapath execution. Always start with fetch, then calculate address, read operands, ALU, write back.

Fetch:

MAR <- PC

IR <- mem[MAR] # or MBR<-mem[MAR]; IR<-MBR

PC <- PC+4

Reg ALU ADD R1, R2, R3:

Read RF: RFOUT1=R1, RFOUT2=R2

A <- RFOUT1

B <- RFOUT2

C <- A+B

RFIN <- C

write RF: R3 <- RFIN

LD DISP(R1), R3:

fetch instruction

Read RF: R1 -> RFOUT1

MAR <- PC

MBR <- mem[MAR] # DISP

PC <- PC+4

A <- RFOUT1

B <- MBR

C <- A+B # effective addr

MAR <- C

MBR <- mem[MAR]

RFIN <- MBR

write RF: R3 <- RFIN

ADD OFF(R1), R2, R3:

fetch; read R1 -> RFOUT1; fetch OFF into MBR; PC+=4

A<-RFOUT1; B<-MBR; C<-A+B; MAR<-C

MBR<-mem[MAR] # mem[OFF+R1]

read R2 -> RFOUT2

A<-MBR; B<-RFOUT2; C<-A+B

RFIN<-C; write RF: R3<-RFIN

ADD D1(R1), D2(R2), R3:

fetch; fetch D1; EA1=R1+D1; MBR=mem[EA1]; TEMP=MBR

fetch D2; EA2=R2+D2; MBR=mem[EA2]

A<-TEMP; B<-MBR; C<-A+B; R3<-C

PUSH Rn: SP<-SP-4; MAR<-SP; MBR<-Rn; mem[MAR]<-MBR

POP Rn : MAR<-SP; MBR<-mem[MAR]; Rn<-MBR; SP<-SP+4

CALL target: fetch target word; PC now return addr;

TEMP<-target; SP-=4; mem[SP]<-PC; PC<-TEMP

RET: MAR<-SP; SP<-SP+4; MBR<-mem[MAR]; PC<-MBR

BR (R4): PC<-R4 value (if memory indirect: MAR<-R4 value; MBR<-mem[MAR]; PC<-MBR)

Pipeline + RISC

Use this when asked: identify hazards, explain split cache, or compare RISC/simple instruction design.

Stages: FI, DI, CO, FO, EI, WO. Modern 5-stage often IF, ID, EX, MEM, WB. Ideal pipeline: each instruction still takes several stages but throughput is 1 instruction/cycle, CPI about 1.

Structural/resource hazard: same hardware needed by two stages; fix by more resources: separate I/D cache, PC incrementer, more buses.

Data hazard: later instruction needs previous result. Main simple pipeline hazard is RAW. Fix by scheduling independent instructions, stalls, or forwarding.

Control hazard: branch/CALL changes PC; next fetch unknown until branch resolved. Branch prediction reduces stalls.

Worked hazard example:

ADD R1, R2, R3

SUB R3, R4, R5

BMI L1

SUB has RAW data hazard because it reads R3 produced by ADD.

BMI depends on SUB flag/result and also causes control hazard because PC may change.

RISC: register-register ALU ops; only LOAD/STORE access memory; fixed/simple formats; few modes; hardware control; pipelining. Tradeoff: maybe more instructions, but lower CPI and clock cycle time.

Assembly Patterns

Use this when asked: write assembly with invented ISA. Adapt the closest loop/function pattern and clearly state registers used for input/output.

Common instruction meanings

MOVl x, Rn # Rn=x

MOVl Ra, Rb # Rb=Ra

LD addr, Rn # Rn=mem[addr]

LD (Ra), Rb # Rb=mem[Ra]

ST Rn, addr # mem[addr]=Rn

ADD Ra, Rb, Rc # Rc=Ra+Rb

SUB Ra, Rb, Rc # Rc=Ra-Rb, sets flags

AND Ra, x, Rb # Rb=Ra & x

BEQ L # branch if zero flag=1

BNE L # branch if zero flag=0

BMI L # branch if negative

for (i=0; i<N; i++) sum += A[i]

MOVl 0, Rsum

MOVl A, Rp

MOVl N, Rn

LOOP: LD (Rp), Rt

ADD Rsum, Rt, Rsum

ADD Rp, 4, Rp

SUB Rn, 1, Rn

BNE LOOP

Use if no MUL: repeated addition.

MYMUL: $R8 \cdot R10 \rightarrow R12$, R10 nonnegative

MYMUL: MOVl 0, R12

MOVl R10, R11

ML: SUB R11, 0, R2 # test counter, set Z

BEQ MDONE

ADD R12, R8, R12

SUB R11, 1, R11

BR ML

MDONE: RET

Use if no DIV or need quotient+remainder.

MYDIV: nonnegative R10/R11 -> quotient R12, remainder R13; R11>0

MYDIV: MOVl 0, R12

MOVl R10, R13

DL: SUB R13, R11, R2

BMI DDONE

MOVl R2, R13

ADD R12, R1, R12

BR DL

DDONE: RET

Use for two 7-segment digits or decimal display.

Convert x<100 to 2 digits, no DIV

input R0; output R1=tens, R2=ones; R3=10

CONV10: MOVl 10, R3

MOVl 0, R1

MOVl R0, R2

C10L: SUB R2, R3, R4

BMI C10D

ADD R1, 1, R1

MOVl R4, R2

BR C10L

C10D: RET

Use for clock/time conversion questions.

CONVHMS: R14 seconds -> R15 h, R16 m, R17 s

CONVHMS:

MOVl R14, R10; MOVl 3600, R11; CALL MYDIV

MOVl R12, R15 # hours

MOVl R13, R10; MOVl 60, R11; CALL MYDIV

MOVl R12, R16; MOVl R13, R17

RET

GETDIGITS: R18 0..99 -> R19 tens, R20 ones

GETDIGITS:

MOVl R18, R10; MOVl 10, R11; CALL MYDIV

MOVl R12, R19; MOVl R13, R20; RET

Programmed I/O

Use this when asked: CSR/status bits, ready/data-ready polling, serial output, rainfall/thermometer/timer display, or "Programmed I/O".

Programmed I/O = CPU controls I/O directly and polls CSR/status bits in a loop. Bad for high performance because CPU busy-waits. Other two techniques: interrupt-driven I/O and DMA.

Generic polling read

READY: LD CSR, R1

AND R1, READYBIT, R2

BEQ READY

MOVl CMBIT, R3

ST R3, CSR

DATA: LD CSR, R1

AND R1, DATABIT, R2

BEQ DATA

LD BR, R0

Serial output flow.

SCSR: err bit0=1, ready bit1=2, write bit2=0

speed115200 bits5..3=100, even bit6=1 -> 0x60.

SUB R1, R1, R1 # index byte

offset, 0..36

READY: LD SCSR, R2

AND R2, 0x2, R3

BEQ READY

WRITE: LD LINE(R1), R6

ST R6, SBRO

MOVl 0x60, R7

ST R7, SCSR # initiate write

LD SCSR, R2

AND R2, 0x1, R3

BNE WRITE # resend if error

AND R2, 0x2, R3

BEQ WAIT

ADD R1, 4, R1

SUB R1, 0x28, R3 # end at 40B

BNE WRITE

HLT

Worked Mini-Answers

Use these as answer-format examples. If a new number appears, follow the same line order: convert -> split fields -> compute final value. For cache examples, keep the block-size arithmetic visible.

Floating Worked Forms

Use these when asked: "write down the bit pattern" or "what value is represented by this hex pattern?"

32-bit custom: S | 10-bit E | 21-bit f, b=2^9-1=511
-15.625: 15.625 = 1111.101b = 1.111101b * 2^3
S=1, E=3+511=514=100000010b
f=1111010000000000000000
bits: 1 100000010 1111010000000000000000
hex: C05E8000

407F8000 in same custom format:
binary = 0100 0000 0111 1111 1000 0000 0000 0000
S=0
E=1000000011b=515 -> actual exponent=4
f=1111100000000000000000
1.f = 1.111111b = 1 + 63/64 = 1.984375
V=1.984375*2^4 = 31.75

30.625 in S|13E|18f, b=4095:
30.625=11110.101b=1.1110101b*2^4
S=0, E=4099=100000000001b
f=1110101000000000000000
bits: 0 100000000011 1110101000000000000

Short Concepts

Use these when asked: short 2-3 mark definitions. Include one precise sentence plus one reason/example.

Direct vs indirect: direct instruction contains operand address; indirect contains address of a memory cell that contains the real operand address.
FIFO: replace block that entered cache earliest.
LRU: replace block unused for longest time.
Memory hierarchy downwards: cost/bit decreases, capacity increases, access time increases, CPU access frequency decreases.
SRAM vs DRAM: SRAM uses logic gates/latches, no refresh, faster, cache. DRAM uses charge, needs refresh, slower, main memory.
Endian: big endian stores most significant byte at lowest address; little endian stores least significant byte at lowest address.
Memory-mapped I/O: I/O registers placed in memory address space; normal load/store read/write CSR/data registers.

Page / TLB Answers

Use this when asked: why virtual memory is useful, what a page fault is, or why TLB helps performance.

Why virtual memory: program sees large logical address space; physical RAM may be smaller; OS/MMU manage movement and protection.
Demand paging: pages loaded only when needed; fast startup; early page faults until working set loaded.
Page fault answer: page not in RAM; OS finds page on disk, finds/free frame, writes dirty victim if needed, updates page tables, restarts faulting instruction/process later.
TLB: cache of page table entries; avoids one extra memory access for page-table lookup on most references.

Cache Worked Forms

Use these when asked: compute number of blocks/sets, set/tag/offset for a specific address, or trace LRU/FIFO.

Example: cache=1024 words, block=64 words, word=4B, 2-way set associative. cache blocks = 1024/64 = 16 sets = 16/2 = 8 block_bytes = 64*4 = 256 = 2^8 offset bits=8, set bits=log2(8)=3

Address C01E in above 2-way cache: block# = 0xC01E / 0x100 = 0xC0 = 192 offset = 0x1E set = 192 mod 8 = 0 tag = floor(192/8)=24=0x18 same block: 0xC000..0xC0FF except 0xC01E

Direct mapped with 16 lines, address E3FC: block_bytes=256, offset=0xFC block#=0xE3 line=0xE3 mod 16 = 3 tag=floor(0xE3/16)=0xE

LRU trace recp table: For each block b: s = b mod sets if b in set s: hit; mark b most recent else miss; if empty insert; else evict least recent in set Keep each set ordered old -> recent.

Disk Worked Forms

Use these when asked: "average rotational latency" or "time for sectors". Convert rpm to ms first.

6000 rpm: rev/s = 6000/60 = 100 one revolution = 1/100 s = 10 ms average rotational latency = 10/2 = 5 ms

7200 rpm: rev/s=120; rev time=1000/120=8.33ms avg latency=4.17ms If 500 sectors/track, sector time=8.33/500=0.0167ms

Cache/VM Analogies

Use this when asked: compare cache and virtual memory. This table gives the paired words.

Cache	Virtual memory
block	page
block offset	page offset
miss	page fault
tag	virtual page number
write-back dirty bit	PTE dirty bit

Performance Comparison

Use when the question compares two processors by GHz. Same clock rate does not mean same speed. Need instruction count, CPI, clock rate, ISA, cache/memory behavior, and processor organization. Formula: CPU time=instruction count*CPI/clock rate.

Datapath Variations

Use this when asked: unfamiliar instruction on the CPU datapath. Translate the instruction meaning first, then move one value at a time.

ST DISP1(R1),DISP2(R3): meaning: mem[R3+DISP2] = mem[R1+DISP1] fetch instr; read R1 -> RFOUT1 MAR<-PC; MBR<-mem[MAR] # DISP1; PC+=4 A<-RFOUT1; B<-MBR; C<-A+B; MAR<-C MBR<-mem[MAR]; TEMP<-MBR # source data MAR<-PC; MBR<-mem[MAR] # DISP2; PC+=4 read R3 -> RFOUT3 # state: add RFOUT3 path if absent A<-RFOUT3; B<-MBR; C<-A+B; MAR<-C MBR<-TEMP; mem[MAR]<-MBR

Direct memory ADD A,B,C (addresses in next words): fetch instruction fetch word Aaddr; MAR=Aaddr; MBR=mem[MAR]; Areg=MBR fetch word Baddr; MAR=Baddr; MBR=mem[MAR]; Breg=MBR Creg=Areg+Breg fetch word Caddr; MAR=Caddr; MBR=Creg; mem[MAR]=MBR

Improve datapath performance: Add multiple buses / separate register-file read buses so two ALU operands move at once. Add direct paths PC<-MAR, MBR->IR, dedicated PC incrementer. In pipeline, split I-cache/D-cache to avoid IF vs data-memory conflict.

Branch / Function

Use these when asked: write if/else, while-loop, odd/even test, or subroutine with saved registers.

max of a[0],a[1] into x
MOVI a,R8 # R8 = base
address of a
LD 0(R8),R9
LD 4(R8),R10
BGT R9,R10,TA # if a[0] > a[1], use a[0]
ST R10,x
BR END
TA: ST R9,x
END: RET

while (temp < 100) { temp += a; ++; }
MOVI 0,R8 # temp
MOVI 1,R9 # a
MOVI 100,R11
W: ADD R8,R9,R8
ADD R9,1,R9
BLT R8,R11,W

odd/even using low bit
AND Rx,1,Rt
BEQ EVEN # zero => even
ODD: ...
EVEN: ...

Function register discipline If subroutine uses registers not declared scratch: push used registers on entry, pop before ret. Return address stored on stack so nested calls work LIFO.

Character Processing

Use this when asked: character conversion or byte-by-byte string processing. ASCII lowercase differs from uppercase by 0x20.

lowercase char in R10 -> uppercase if a..z
PUSH R8; PUSH R9
MOVI 0x61,R8 # 'a'
MOVI 0x7a,R9 # 'z'
BLT R10,R8,RET1
BGT R10,R9,RET1
SUB R10,0x20,R10 # 'a'-'A'
RET1: POP R9; POP R8; RET

I/O Variations

Use this when asked: a sensor plus LEDs, or any CSR with ready/start/data-ready bits. Poll ready, command device, poll data, read buffer, display.

Sensor + 2 LEDs. Set masks from the question.
Thermometer exercise: CSR=TCSR, BR=TBR, READY=1, START=2, DATA=4
Rainfall exercise: CSR=RFCR, BR=RFBR, READY=1, START=4, DATA=2
READY: LD CSR,R2
AND R2,READY,R3
BEQ READY
MOVI START,R1
ST R1,CSR
LD CSR,R2
AND R2,DATA,R3
BEQ LOOP
LD BR,R0
CALL CONV10 # R1=tens,
R2=ones ST R1,LEDBR1 # tens/left
digit ST R2,LEDBR2 # ones/right
digit

If control bit should be set while preserving CSR:
LD CSR,R1
OR R1,CMDBIT,R1
ST R1,CSR

If command register expects command value:
MOVI CMDBIT,R1
ST R1,CSR

In answer, state assumption if ambiguous.

For CSR bit masks, convert bit n to value 2^n. Example serial-control format: bit6 even parity=64, speed 115200 in bits5..3 is 100 = bit5 = 32, write bit2=0, so control 64+32=96=0x60.

Lower-Priority Topics

Use this to prioritize review: if a topic below appears only indirectly, cover it only when the wording explicitly asks.

Lower priority unless explicitly asked: proof that n*m multiplication has at most n+m bits, full proof of 2's-complement multiplication, Booth tables, S-R latch, detailed 2-to-1 multiplexer, IEEE NaN/infinity/subnormal rules, full simulator implementation details, full RAID 2 mechanics.

Answer Phrases

Use these when asked: conceptual "explain why" questions. Select the relevant idea and add one course keyword.

Programmed I/O bad: CPU repeatedly checks status in a loop and wastes cycles while device is slow.
Interrupt advantage: CPU can execute other code until device signals completion.
DMA advantage: large block transfer directly between I/O controller and memory with little CPU involvement.
Split cache hazard: avoids structural conflict when pipeline fetches instruction and accesses data in same cycle.
Direct map pro/con: simple and fast; conflict misses if active blocks map to same line.
Fully assoc pro/con: flexible and higher hit ratio; must compare many tags.
Set assoc: compromise between direct and fully associative.
Floating-point caveat: fixed total bit patterns, so more range means less precision; numbers are not evenly spaced.

Final Checklist

Before moving on: did you write units? For cache, did you say byte/word addressable? For assembly, did you state input/output registers and assumptions? For invented ISA questions, correctness and comments matter more than exact syntax.

Encoding Mini-Patterns

Use this when asked: hand assemble instruction-encoding exercises. Replace register numbers and next-word addresses carefully.

Instruction word = op s1 s2 d, each field 1 byte.
ADD R4,R1,R4 -> 00 04 01 04 = 00040104
SUB R3,R1,R5 -> 01 03 01 05 = 01030105
MOV R1,R2 -> 05 01 00 02 = 05010002
LD P1,R1 -> 06 00 FF 01 [addr(P1)]
ST R4,P -> 07 04 FF 00 [addr(P)]
BNZ L -> 08 02 FF 00 [addr(L)]
HLT -> 09 00 00 00 = 09000000
PUSH R12 -> 0A 0C 00 00 = 0A0C0000
POP R13 -> 0B 00 00 0D = 0B00000D
CALL SQ -> 0C 00 FF 00 [addr(SQ)]
RET -> 0D000000

Arithmetic Extras

Use this when asked: relative error, multiplication proof idea, or product bit bound. These are lower-frequency but cheap marks if they appear.

Relative error: if exact value is A and stored value is A', |A-A'|/|A| (signed form only if asked). For non-terminating binary fractions, write enough repeating bits, truncate/round, reconstruct A', then compute.
2's multiplication idea: sign-extend partial products to double precision; for multiplier sign bit 1 add two's complement of multiplicand; ignore carry out. Avoid proof unless asked.
Unsigned product bound: n-bit max times m-bit max is (2^n-1)(2^m-1) < 2^(n+m), so product needs at most n+m bits.

OS / Protection

Use this when asked: user/kernel mode, system calls, protected resources, or time sharing.

User mode cannot directly use protected resources or switch itself to kernel mode. System calls enter OS at controlled entry points and run in kernel/supervisor mode. Time sharing: when a process blocks on I/O or uses up its time slice, OS schedules another process.

More Worked Quickies

Use these when asked: common worked-number formats from practice exercises. Follow the line order, not just the final answer.

8-bit reps of 24 and -24:
+24 unsigned = 00011000
1's +24 = 00011000
2's +24 = 00011000
Excess-63 +24: 24+63=87 = 01010111

-24:
start +24 = 00011000
1's comp = 11100111
2's comp = 11101000
Excess-63: -24+63=39 = 00100111

32-bit single-like S|8E|23f, b=127:
13.375 = 1101.011b = 1.101011b*2^3
S=0, E=3+127=130=10000010b
f=1010110000000000000000
bits=010000010101110000000000000000
hex=4156000

Address C912F, cache 1024 words, block 128 words, 2-way, word=4B: block_bytes=512=2^9, offset=0x12F blocks=1024/128=8, sets=8/2=4 block#=floor(0xC912F/0x200)=0x648 set=0x648 mod 4 = 0 tag=floor(0x648/4)=0x192 same block: 0xC9000..0xC91FF

Spatial locality code: in row-major arrays, use for i rows outside and for j cols inside so a[i][j] touches consecutive memory.
RAID 5 capacity: with N equal disks, usable capacity=(N-1)*disk. Example six 10T disks -> 50T usable.

VM Address Calc

Use this when asked: translate a virtual/logical address using a page table or TLB.

page_size = 2^p bytes -> offset has p bits
VPN = floor(VA/page_size)
offset = VA mod page_size
If PTE valid: PA = PFN*page_size + offset
If PTE invalid: page fault; OS loads page, updates PTE/TLB, restarts.